Arquitectura de Computadores

Práctica 1 — Guía de Defensa Oral

El Gran Relato de la Jerarquía de Memoria

Un corredor de élite, su equipo de avituallamiento
y cuatro entrenadores telépatas

Contenido de este documento

- Cómo se descubren los parámetros del hardware (`getconf`)
- Analogía completa: pista, baldosas, zancadas y cronómetro
- Detalles clave del código C
- Interpretación de las cuatro gráficas con datos reales
- Resumen ejecutivo en un minuto
- 5 preguntas trampa con respuestas blindadas

FinisTerrae III (CESGA) · Intel Xeon Platinum 8352Y

March 4, 2026

Contents

1 ¿Cómo descubrimos las medidas de la pista? 2

2 La Pista y las Estaciones de Agua 2

3 El Equipo de Avituallamiento 3

4 La Zancada y los Pasos (Stride D y R) 3

5 Las Zapatillas del Corredor (Double vs Int) 4

6 El Despacho del Manager (Detalles del Código) 4
- 6.1 Alinear la pista correctamente (`aligned_alloc`) 4
- 6.2 Inicializar con valores en $[1, 2)$ con signo aleatorio 4
- 6.3 Imprimir S[] al terminar 5

7 El Cronómetro (Ciclos y `counter.h`) 5

8 El Entrenador Telépata (El Prefetcher Hardware) 5

9 Lo que nos dicen las gráficas 6
- 9.1 `grafica_memoria.png` — Double indirecto, 5 strides 6
- 9.2 `comparativa_D1.png` — Stride D=1, tres variantes 6
- 9.3 `comparativa_D16.png` — Stride D=16, tres variantes 7
- 9.4 `comparativa_experimentos.png` — Tres paneles 7

10 Resumen en un Minuto 8

11 Preguntas Trampa y Cómo Responderlas 8

1 ¿Cómo descubrimos las medidas de la pista?

Antes de correr la carrera, necesitamos saber exactamente cómo es la pista. En el superordenador FinisTerrae III ejecutamos:

```
getconf -a | grep CACHE
```

Listing 1: Consulta de parámetros de caché al sistema operativo

Este comando le pregunta directamente al sistema operativo Linux los parámetros físicos del procesador. La salida más relevante fue:

```
LEVEL1_DCACHE_SIZE        49152    -> 48 KB  (L1d)
LEVEL1_DCACHE_LINESIZE    64       -> 64 bytes por linea
LEVEL2_CACHE_SIZE         1310720  -> 1.25 MB (L2)
LEVEL3_CACHE_SIZE         50331648 -> 48 MB  (L3)
```

Listing 2: Salida real del FinisTerrae III

Con estos datos calculamos los valores de S (número de líneas de caché que caben en cada nivel):

$$S_1 = \frac{48\,\text{KB}}{64\,\text{bytes}} = 768 \text{ líneas} \tag{1}$$

$$S_2 = \frac{1{,}25\,\text{MB}}{64\,\text{bytes}} = 20\,480 \text{ líneas} \tag{2}$$

$$S_3 = \frac{48\,\text{MB}}{64\,\text{bytes}} = 786\,432 \text{ líneas} \tag{3}$$

¿Por qué es crítico obtenerlos así?

Estos tres valores son el esqueleto de todo el experimento. Si nos equivocáramos aquí, los valores de L no caerían en los niveles correctos y las gráficas no mostrarían las transiciones reales. Es imprescindible obtenerlos directamente del hardware y no suponerlos.

2 La Pista y las Estaciones de Agua

Analogía: Líneas de Caché y el parámetro L

La Línea de Caché (64 bytes): Imagina que la pista de atletismo está dividida en baldosas gigantes, y cada baldosa mide exactamente 64 metros (bytes). Cada vez que el corredor pisa una baldosa nueva, necesita agarrar una bandeja entera de agua — una línea de caché.

El parámetro L (Líneas): Es la **longitud total de la carrera**. Si L es pequeño (ej. 384 líneas), es una carrera cortita de barrio. Si L es enorme (ej. 3 145 728 líneas), es una ultramaratón que cruza todo el país. El parámetro L determina en qué nivel de la jerarquía van a estar los datos durante el experimento.

3 El Equipo de Avituallamiento

El atleta es rapidísimo, pero no lleva el agua encima. Depende de su equipo, que está a diferentes distancias:

Los cuatro niveles de la jerarquía

L1 — La mochila de hidratación: Está pegada al corredor. Le caben pocos datos (48 KB), pero tarda poquísimo en beber: apenas **∼5 ciclos**.

L2 — El entrenador en bicicleta: Va justo al lado de la pista. Tiene más agua (1,25 MB), pero el corredor tarda **∼14 ciclos** en agarrar la botella.

L3 — La furgoneta de apoyo: Está aparcada fuera de la pista. Tiene mucha agua (48 MB), pero ir hasta ella frena al corredor **∼40–50 ciclos**.

RAM — El almacén de la fábrica: Es gigantesco (256 GB), pero está en otra ciudad. Esperar agua de la RAM congela al corredor **∼150–200 ciclos**.

4 La Zancada y los Pasos (Stride D y R)

Analogía: el stride y el número de accesos

El Stride (D) es la **longitud de la zancada** del atleta:

- Si $D = 1$, el corredor da pasitos muy cortos y pisa todos los elementos de la baldosa.
- Si $D = 16$ o $D = 64$, el corredor va dando saltos gigantes.

La cantidad de elementos (R) es el **número total de pisadas** en toda la carrera:

$$R = \frac{L \times \text{CLS}}{D \times \text{sizeof(tipo)}}$$

El Salto de Caché ocurre cuando la zancada cruza la frontera de los 64 bytes y pisa una baldosa nueva. En ese momento el equipo tiene que traer una bandeja de agua nueva.

5 Las Zapatillas del Corredor (Double vs Int)

Analogía: el tipo de dato como equipamiento

Double (8 bytes) — Botas pesadas: En cada baldosa de 64 bytes solo caben **8 pisadas** antes de saltar a la siguiente.

Int (4 bytes) — Zapatillas de clavos: Ocupan la mitad de espacio; en la misma baldosa el atleta puede dar **16 pisadas**. Como da más pasos aprovechando la misma bandeja de agua, el rendimiento mejora drásticamente (hasta un 43% en zonas RAM).

6 El Despacho del Manager (Detalles del Código)

Antes de que el corredor salga a pista, el manager del equipo — **nuestro programa en C** — tiene que preparar tres cosas esenciales.

6.1 Alinear la pista correctamente (`aligned_alloc`)

```c
double *A = (double*)aligned_alloc(CLS, N * sizeof(double));
```

Listing 3: Reserva alineada al inicio de línea de caché

Le pedimos al SO que el vector `A[]` empiece exactamente al inicio de una baldosa (línea de caché de 64 bytes), no en medio. Si empezara en medio, el primer acceso desperdiciaría parte de la línea y las medidas de L1 serían imprecisas. Sin alineación, estaríamos midiendo a medio gas desde la salida.

6.2 Inicializar con valores en $[1, 2)$ con signo aleatorio

```c
double val = 1.0 + (double)rand() / RAND_MAX;
if (rand() % 2) val = -val;
A[ind[i]] = val;
```

Listing 4: Inicialización anti-desbordamiento y calentamiento real

Dos razones para este rango específico:

- **Calentamiento real:** inicializar todos los datos que luego se leerán garantiza que las TLBs y cachés ya conocen esas direcciones. Con cero o valor fijo, el procesador podría optimizar de formas que no reflejan el acceso real.
- **Anti-desbordamiento:** los valores en $[1, 2)$ con signo aseguran que la suma acumulada de R elementos nunca produce `Inf` o `NaN`, lo que falsearía el resultado y podría hacer que el compilador eliminase el bucle.

6.3 Imprimir S[] al terminar

```
printf("Resultados S[]:");
for (int k = 0; k < REPS; k++) printf(" %.6f", S[k]);
```

Listing 5: Impresión de resultados: anti-optimización del compilador

El enunciado exige guardar el resultado de cada repetición en un vector `S[]` e imprimirlo. El propósito técnico es claro: **evitar que el compilador elimine el bucle por completo**. Si la suma no se guarda ni se usa, `gcc -O0` puede detectar que el bucle es *trabajo muerto* y no ejecutarlo. Al imprimir `S[]`, obligamos a que el resultado exista físicamente en memoria.

7 El Cronómetro (Ciclos y `counter.h`)

Los **ciclos** son los segundos que marca el cronómetro del entrenador. La librería `counter.h` usa la instrucción ensamblador `RDTSC` (*Read Time-Stamp Counter*), que lee directamente el contador de ciclos del procesador sin pasar por el sistema operativo:

```
start_counter();            // Pulsar START
for (int k = 0; k < 10; k++) {
    for (long long i = 0; i < R; i++)
        suma += A[ind[i]];
    S[k] = suma;
}
double ciclos_totales = get_counter();     // Pulsar STOP
double ciclos_por_acceso = ciclos_totales / ((double)R * 10);
```

Listing 6: Estructura de medición con 10 repeticiones internas

- Se hacen **10 repeticiones internas** y se divide entre $R \times 10$ para obtener el coste medio por acceso individual.
- Externamente se repite el experimento **10 veces** y se toma la **media geométrica de los 3 mejores** valores, eliminando el ruido del planificador del SO.

Interpretación del resultado

Si el valor es ∼7 ciclos (verde), el corredor voló. Si el valor es ∼18 ciclos (rojo), el corredor se quedó parado esperando el agua desde la RAM.

8 El Entrenador Telépata (El Prefetcher Hardware)

El procesador tiene cuatro *prefetchers* que intentan adivinar dónde va a pisar el corredor para ponerle el agua en la mano *antes* de que la pida.

Caso ideal: D=1, corredor predecible

El entrenador telépata lee sus movimientos perfectamente. Incluso en una ultramaratón (RAM profunda de 192 MB), trae el agua desde el almacén con antelación. El corredor marca **∼7.2 ciclos**, como si nunca hubiera salido de la mochila L1.

Caso peligroso: D=16, prefetch pollution

El atleta da saltos de exactamente **128 bytes** (16×8 bytes/double). El *L2 Spatial Prefetcher* tiene una manía: siempre que pides una línea de 64 bytes, trae automáticamente la línea contigua formando un bloque de 128 bytes. Resultado: el programa salta exactamente por encima de la línea que el prefetcher está trayendo gratis. Ese trabajo extra es **prefetch pollution** y el tiempo se hunde hasta los **17.09 ciclos en RAM**.

9 Lo que nos dicen las gráficas

9.1 `grafica_memoria.png` — Double indirecto, 5 strides

Esta gráfica muestra ciclos/acceso (eje Y) frente al número de líneas L en escala logarítmica (eje X). Se pueden leer tres regiones muy distintas:

Región plana izquierda ($L < 20\,480$): Todas las curvas caminan casi juntas por debajo de 8 ciclos. Los datos caben en L1 o L2 y el prefetcher oculta las latencias. $D = 1$ va pegado al suelo (∼7.1 ciclos). $D = 512$ arranca un poco más alto en $L = 384$ (∼8.88 ciclos) por el overhead del bucle con muy pocos elementos, pero se normaliza rápidamente.

Región de transición ($524\,288 < L < 786\,432$): Aquí está el salto dramático que valida experimentalmente $S_3 = 786\,432$. Para $D \geq 16$ las curvas se disparan verticalmente entre 32 y 48 MB de footprint. $D = 1$ sigue completamente plano; $D = 8$ sube suavemente hasta ∼9.4 ciclos.

Región RAM ($L > 786\,432$): Las curvas se separan definitivamente. $D = 1$ se queda congelado en 7.2 ciclos. $D = 8$ sube hasta ∼9.8 ciclos. $D = 16$, $D = 64$ y $D = 512$ convergen entre 16 y 18 ciclos y se estabilizan: la diferencia entre 96 MB y 192 MB es inferior al 2%.

9.2 `comparativa_D1.png` — Stride D=1, tres variantes

Esta gráfica ilustra el poder absoluto del prefetcher. Las tres líneas son **completamente planas en toda la escala**, desde L1 hasta 192 MB de RAM. La única diferencia es el nivel en el que se estabilizan:

- **Int indirecto (∼6.8 ciclos):** más bajo porque `ADD` entero tiene menor latencia que `FADD` flotante.

- **Double directo (∼7.1 ciclos):** ligeramente por debajo del indirecto porque no hay que leer `ind[]`.
- **Double indirecto (∼7.2 ciclos):** el experimento base.

Conclusión de esta gráfica

Con $D = 1$ el hardware hace trampa y oculta completamente la jerarquía de memoria.

9.3 `comparativa_D16.png` — Stride D=16, tres variantes

Esta es la gráfica más informativa del experimento porque separa dramáticamente las tres variantes:

- **Double indirecto (azul):** peor caso absoluto, **17.09 ciclos en RAM**. Sufre doble penalización: *prefetch pollution* de $D = 16$ sobre `A[]` más la contención de `ind[]` compitiendo por el mismo ancho de banda.
- **Double directo (verde punteado):** mejora ∼1.2 ciclos en RAM (15.86 vs 17.09). Sin `ind[]`, la pollution afecta solo a `A[]` y hay menos tráfico compitiendo por el bus.
- **Int indirecto (naranja):** el gran sorprendente, **solo 9.79 ciclos en RAM**, mejora del 43% respecto al double indirecto. Con tipo `int`, R se duplica (más accesos por línea), pero el footprint en caché es el mismo, por lo que la pollution no se duplica. La relación "trabajo hecho / basura traída" mejora drásticamente.

La escalada visible a partir de $L = 524\,288$ es exactamente el momento en que los datos desbordan L3 y empiezan a llegar desde RAM.

9.4 `comparativa_experimentos.png` — Tres paneles

Los tres paneles muestran las cinco curvas de stride en cada variante. Los patrones comunes refuerzan que los resultados son reproducibles y coherentes:

- En los tres paneles, $D = 1$ (azul) es siempre la línea plana inferior.
- En los tres paneles, el "salto" ocurre siempre en el mismo L (∼524 288), confirmando que $S_3 = 786\,432$ es correcto independientemente de la variante.
- Las líneas verticales naranjas (límite L3) coinciden exactamente con donde las curvas se doblan, validando los valores obtenidos con `getconf`.

10 Resumen en un Minuto

La gran conclusión de la práctica

El rendimiento de la memoria no es un límite físico fijo, sino el resultado de cómo nuestro programa dialoga con las predicciones del procesador.

En nuestro experimento, la gran revelación fue descubrir que **el stride $D = 1$ es completamente inmune a la jerarquía de memoria**. Cuando el atleta corre con pasos cortos y 100% predecibles, su *entrenador telépata* —los cuatro prefetchers hardware trabajando en equipo— es tan brillante que oculta todas las latencias. No importa si la carrera llega hasta los 192 MB de RAM; el tiempo se congela en **∼7.2 ciclos**, idéntico a si los datos estuvieran en la caché L1.

Sin embargo, si el atleta da saltos intermedios de exactamente 128 bytes ($D = 16$), el *L2 Spatial Prefetcher* entra en pánico y genera **prefetch pollution**, llenando la pista de bandejas de agua inútiles que hunden el tiempo hasta los **17.09 ciclos** en RAM.

En definitiva, la jerarquía no es una escalera estricta. Gracias al hardware prefetcher, la penalización real desde L1 hasta RAM se atenúa en un factor de **14 veces**, pero *solo* si programamos a favor de sus predicciones.

11 Preguntas Trampa y Cómo Responderlas

Pregunta Trampa 1 — Valores de S1, S2 y S3

"¿Cómo supisteis los valores de S1, S2 y S3 del procesador del CESGA?"

Respuesta: Ejecutamos `getconf -a | grep CACHE` directamente en el nodo del FinisTerrae III. Este comando consulta los parámetros del SO Linux sobre la arquitectura hardware. Nos devolvió los tamaños exactos de cada nivel en bytes: 49 152 para L1d (48 KB), 1 310 720 para L2 (1,25 MB) y 50 331 648 para L3 (48 MB). Dividiendo cada valor entre 64 bytes (confirmado por `LEVEL1_DCACHE_LINESIZE`) obtuvimos $S_1 = 768$, $S_2 = 20\,480$ y $S_3 = 786\,432$. Es esencial obtenerlos directamente del hardware porque determinan la calidad de todo el experimento.

Pregunta Trampa 2 — `#SBATCH -c 64`

"Estás reservando 64 cores pero el código es secuencial. ¿No es un desperdicio enorme?"

Respuesta: Puede parecer un desperdicio, pero es absolutamente esencial para la fiabilidad del experimento. Al reservar el nodo completo con `-c 64`, impedimos que Slurm meta procesos de otros usuarios en el mismo nodo físico. Si otro usuario

estuviera ejecutando algo, su proceso ensuciaría la caché L3, que es compartida por todo el socket. Para medir latencias de memoria con precisión de ciclos, necesitamos un entorno completamente aislado.

Pregunta Trampa 3 — `gcc -O0`

"¿Por qué compilasteis con `gcc -O0` y no con `-O3`?"

Respuesta: Porque nuestro objetivo no es que el programa sea rápido, sino medir la latencia pura del hardware de memoria. Con `-O3`, el compilador podría vectorizar el bucle con instrucciones AVX, desenrollarlo, o incluso eliminarlo entero al detectar que la suma no se usa fuera del bucle. Con `-O0` garantizamos que por cada iteración se hace exactamente la petición de memoria que queremos medir, sin enmascarar ni alterar los accesos.

Pregunta Trampa 4 — ¿Habéis llegado a RAM pura?

"¿Cómo podéis estar seguros de que a partir de $L = 1\,572\,864$ habéis llegado a la RAM pura?"

Respuesta: Lo sabemos por dos razones combinadas. Primero, el tamaño del footprint: la L3 tiene 48 MB ($S_3 = 786\,432$ líneas). Para $L = 1\,572\,864$ el footprint es de 96 MB, y para $L = 3\,145\,728$ es de 192 MB; ambos desbordan masivamente la L3. Segundo, la estabilización: entre los 96 MB y los 192 MB la diferencia de ciclos es inferior al 2% en todos los strides. Esta falta de degradación adicional confirma que hemos tocado el fondo de la jerarquía, donde la latencia es constante.

Pregunta Trampa 5 — ¿Por qué D=16 es peor que D=64?

"¿Por qué D=16 es peor que D=64 o D=512 en la zona RAM, si tiene un stride más pequeño?"

Respuesta: Es la pregunta clave del experimento. Con $D = 16$ y tipo `double`, la distancia entre accesos consecutivos es exactamente **128 bytes** (16×8 bytes), que coincide con el tamaño del bloque del *L2 Spatial Prefetcher*. Este prefetcher trae automáticamente parejas de líneas de 64 bytes. Con $D = 16$, nuestro programa salta exactamente por encima de cada segunda línea que el prefetcher trae: trabaja el doble trayendo basura, satura el bus de memoria y expulsa datos útiles de la caché. Con $D = 64$ o $D = 512$ el stride es tan grande que el prefetcher simplemente no puede predecir nada y se queda quieto, eliminando también la pollution. Por eso $D = 64$ y $D = 512$ *no son peores* que $D = 16$; de hecho, en RAM son ligeramente mejores.